

Steering the ship: **Kubernetes & OpenShift**, explained from zero.

Kubernetes is Greek for *helmsman* – the person who steers a ship. This guide starts before the ship even exists: why we containerize software at all, then builds up every concept, one clear diagram at a time, until OpenShift's enterprise deck makes sense too.

Audience • complete beginners

Covers • K8s core + OpenShift

Format • concepts + diagrams

No setup required

PART 00 **Why does orchestration exist?**

Before you can understand Kubernetes, you need to understand the problem it was invented to solve. That problem is not "how do I run one program" – it's "how do I run *hundreds* of programs, reliably, across many machines, forever."

Imagine you built a web app. On your laptop it's a single process. Easy. Now the real world arrives:

- **Traffic grows.** One copy of your app can't handle it. You need ten copies – but who starts them, and on which machines?
- **Machines die.** A server crashes at 3 a.m. Something must notice and restart your app somewhere healthy – without waking you up.
- **You ship updates.** You want to replace v1 with v2 without any downtime, and roll back instantly if v2 misbehaves.
- **Copies must find each other.** Your web app needs to reach your database, even as copies come and go and their addresses change.

Doing all of this by hand – SSH-ing into servers, starting processes, editing load-balancer configs – is slow, error-prone, and impossible at scale. **Orchestration** is the practice of handing those chores to software. You describe the *desired state* ("I want 10 healthy copies of v2, reachable at this address"), and the orchestrator continuously makes reality match that description.

THE MENTAL MODEL

An orchestrator is a **tireless operations team that never sleeps**. You don't give it step-by-step instructions ("start this, then that"). You give it a goal, and it figures out the steps "and keeps re-checking, forever, whether the goal is still being met."

Kubernetes is the most widely used orchestrator in the world. OpenShift is a commercial, batteries-included *distribution* of Kubernetes built by Red Hat. Everything in Parts 01"07 is pure Kubernetes; Parts 08"09 show what OpenShift layers on top.

PART 01 Containers vs. virtual machines

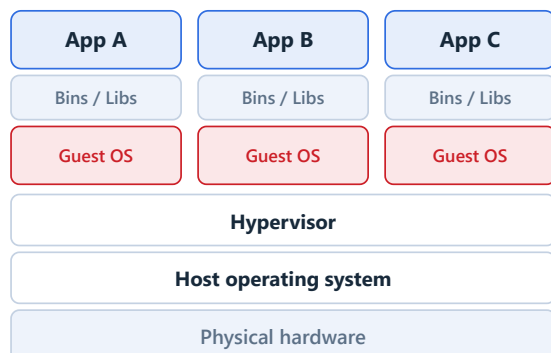
Kubernetes orchestrates *containers*, so first: what is a container, and why not just use virtual machines?

A **container** packages your application *together with* everything it needs to run – the code, the runtime, the libraries, the system tools – into one standardized, portable unit. The promise is simple: *it runs the same on your laptop, in a test cluster, and in production*, because the environment travels with the app.

A **virtual machine (VM)** also isolates software, but it does so by emulating an entire computer, including a full *guest operating system*. That's heavy: gigabytes of disk, minutes to boot. A container skips the guest OS entirely and shares the host's kernel – so it's typically megabytes in size and starts in a fraction of a second.

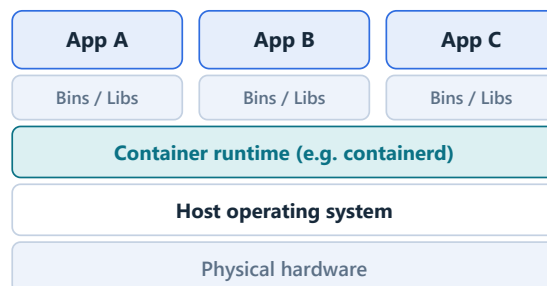
Virtual machines

each VM ships a whole guest OS – heavy & slow



Containers

share one host OS kernel – light & fast



Notice: no guest OS layer. That's the whole difference.

Figure 1 – Containers vs. VMs. A VM virtualizes hardware and runs a full guest OS per app. A container virtualizes the *operating system*, sharing the host kernel, so it's far smaller and faster to start. Kubernetes schedules containers, not VMs.



Container vs. image. A container *image* is the frozen blueprint (built once, e.g. with a `Dockerfile`, stored in a *registry*). A *container* is a running instance of that image. Same relationship as a class and an object, or a recipe and a cooked meal.

PART 02 What Kubernetes actually is

Kubernetes (often shortened to **K8s** â€” "K", eight letters, "s") is an open-source platform for running containers across a fleet of machines. Google open-sourced it in 2014; it's now stewarded by the Cloud Native Computing Foundation.

The single most important idea in Kubernetes is **declarative, desired-state management**. You never tell Kubernetes *how* to do things step by step. Instead you write a document â€” usually YAML â€” that describes *what you want to be true*:

● A TASTE OF KUBERNETES YAML â€” "THE DESIRED STATE"

```
# "I want 3 healthy copies of my web app running, always."
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3           # ← the goal: three copies
  template:
    spec:
      containers:
        - name: web
          image: my-web-app:2.0
```

You hand this to the cluster. From that moment on, Kubernetes runs an endless loop comparing *desired state* ("3 copies") against *actual state* ("how many are really running?"). If a copy dies, actual drops to 2, the loop notices the gap, and Kubernetes starts a new one â€” no human involved. This loop is called the **reconciliation loop**, and it is the beating heart of the entire system.

THE MENTAL MODEL

Think of a **thermostat**. You set it to 21Â°C (desired state). It constantly reads the room (actual state) and turns heating on or off to close the gap. You never micromanage the furnace. Kubernetes is a thermostat for your applications.

What Kubernetes gives you, out of the box:

- **Self-healing** “ restarts failed containers, replaces and reschedules them when nodes die.
- **Horizontal scaling** “ run more or fewer copies with one command, or automatically based on load.
- **Service discovery & load balancing** “ gives your containers a stable address and spreads traffic across copies.
- **Automated rollouts & rollbacks** “ ship new versions gradually; revert instantly if something breaks.
- **Configuration & secret management** “ inject settings and credentials without rebuilding images.

PART 03 Cluster architecture

A Kubernetes **cluster** is a set of machines ("nodes") working as one. Those nodes fall into two roles: the **control plane** (the brain that makes decisions) and the **worker nodes** (the muscle that runs your containers).

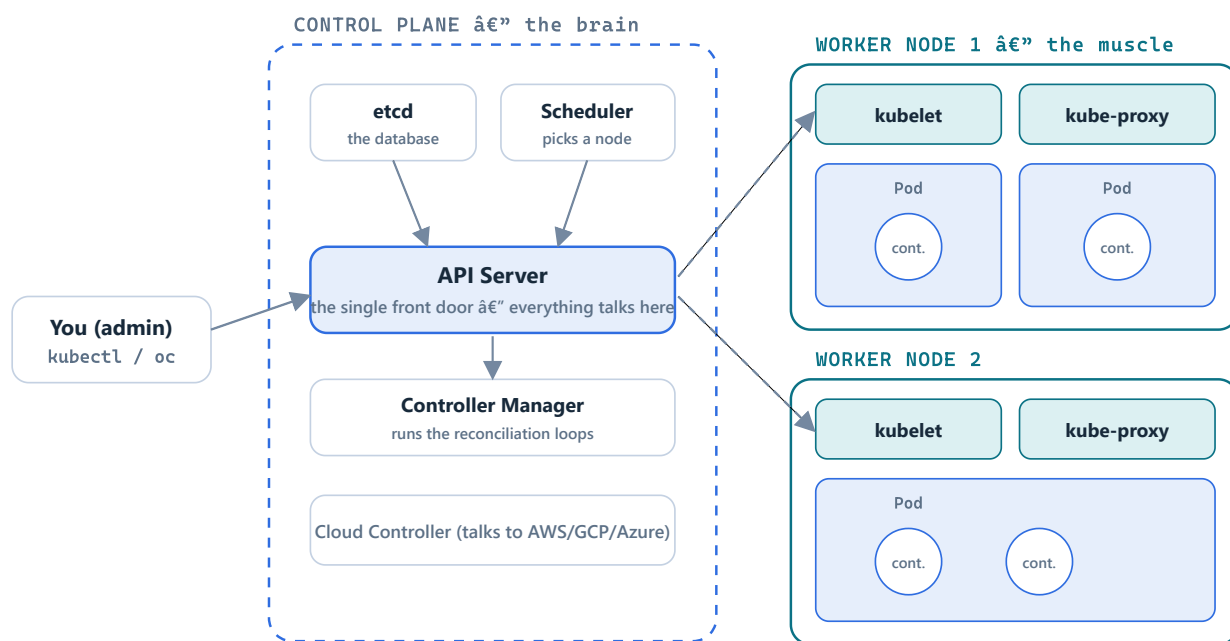


Figure 2 â€” Cluster anatomy. You send your desired state to the **API Server**, the control plane's single front door. It's stored in **etcd**, the **Scheduler** assigns work to nodes, and **controllers** keep reconciling. On each worker node, the **kubelet** starts the containers and **kube-proxy** wires up networking.

The control plane, piece by piece

API Server **core**

The front door. Every command, from you or from internal components, goes through it. It validates requests and is the *only* thing that reads/writes etcd.

etcd **store**

A reliable key-value database holding the entire cluster state â€” the single source of truth. Lose etcd and you lose the cluster's memory.

Scheduler core

When a new Pod needs a home, the scheduler picks the best node for it based on available CPU/memory, rules, and constraints.

Controller Manager core

Runs the reconciliation loops. The "Deployment controller," "Node controller," etc. all watch state and nudge reality toward desired.

The worker nodes, piece by piece

kubelet agent

The control plane's agent on every node. It receives Pod specs and makes sure those containers are actually running and healthy.

kube-proxy net

Maintains the network rules that let traffic reach the right Pods, wherever they live in the cluster.

Container runtime exec

The software that actually pulls images and runs containers (e.g. `containerd` or `CRI-O`). Kubernetes tells it what to run.



You almost never touch these components directly. Managed services (Amazon EKS, Google GKE, Azure AKS) and OpenShift run the control plane for you. Understanding the parts still matters – it's what makes error messages and behavior make sense.

PART 04 The core objects you'll use every day

Kubernetes gives you building blocks called **objects** (or *resources*). You compose them to describe your application. Here are the ones that matter, built up from the smallest.

The Pod “the atom of Kubernetes

A **Pod** is the smallest thing Kubernetes runs. It's a wrapper around *one or more* containers that are so tightly related they must live together “ they share the same network address and can share storage. Most Pods hold exactly one container; a second "sidecar" container is added only when it needs to run right next to the main one.



Pods are cattle, not pets. They are deliberately *disposable*. A Pod can be killed and replaced at any moment, and its replacement gets a new IP address. You never rely on a specific Pod surviving “ that's exactly why the higher-level objects below exist.

ReplicaSet & Deployment “ keeping copies alive

Because Pods are disposable, you rarely create one directly. Instead you create a **Deployment**. A Deployment manages a **ReplicaSet**, and the ReplicaSet's only job is to keep the requested number of identical Pods running. Ask for 3, and if one dies, the ReplicaSet immediately makes a fourth to get back to 3.

The Deployment adds the thing you actually want for real apps: **controlled updates**. Change the image from `v1` to `v2`, and the Deployment rolls out new Pods gradually while retiring old ones “ and can roll back if the new version fails.

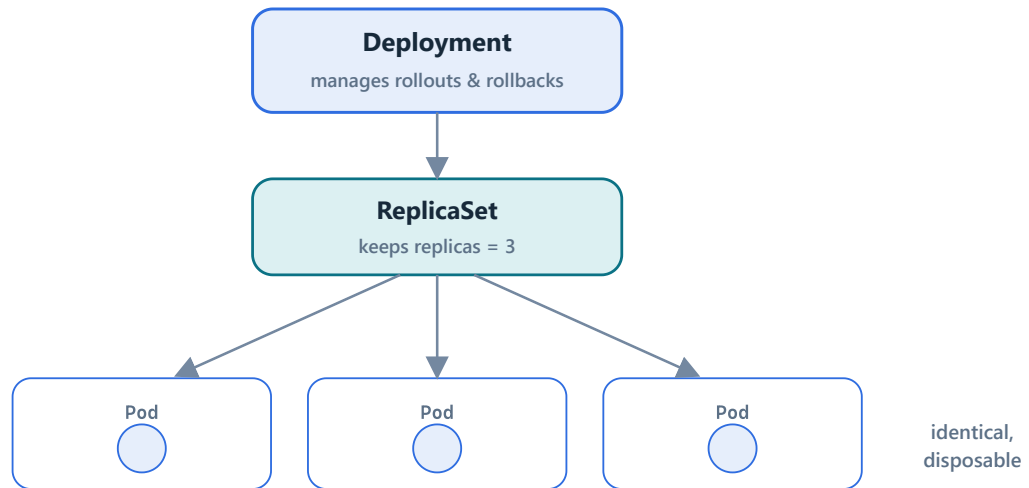


Figure 3 – The workload hierarchy. You declare a **Deployment**; it creates a **ReplicaSet**; the ReplicaSet guarantees the right number of **Pods**. Kill any Pod and a replacement appears. Update the Deployment and it rolls Pods over safely.

Other ways to run workloads

StatefulSet workload

Like a Deployment but for apps that need *stable identity* and storage – databases, message queues. Pods get sticky names and persistent disks.

DaemonSet workload

Runs exactly one copy of a Pod on *every* node. Perfect for log collectors or monitoring agents.

Job workload

Runs a Pod to completion once – a batch task, a migration – then stops. It's done when the work is done.

CronJob workload

A Job on a schedule. "Run this backup every night at 2 a.m." Same idea as Unix cron.

Namespaces – dividing the cluster into rooms

A **Namespace** is a virtual partition inside a cluster. It lets many teams or environments (dev, staging, prod) share one physical cluster without stepping on each other – names must be unique only *within* a namespace, and you can apply quotas and access rules per namespace. (In OpenShift, a namespace with extra policy attached is called a **Project** – more on that later.)

PART 05 Networking: how traffic finds your Pods

Pods are disposable and their IP addresses change constantly. So how does anything reliably reach them? The answer is the **Service** – a stable front for a changing set of Pods.

A **Service** gives a group of Pods (selected by their labels) one unchanging virtual address and name. Traffic sent to the Service is automatically load-balanced across whichever Pods are currently healthy. Pods appear and disappear behind it; the Service address never changes.

THE MENTAL MODEL

A Service is like a **restaurant's phone number**. Staff (Pods) are hired and quit constantly, but you always call the same number and reach *whoever is working right now*. You never need to know individual staff members' personal numbers.

Services come in types, layered from "internal only" to "exposed to the world":

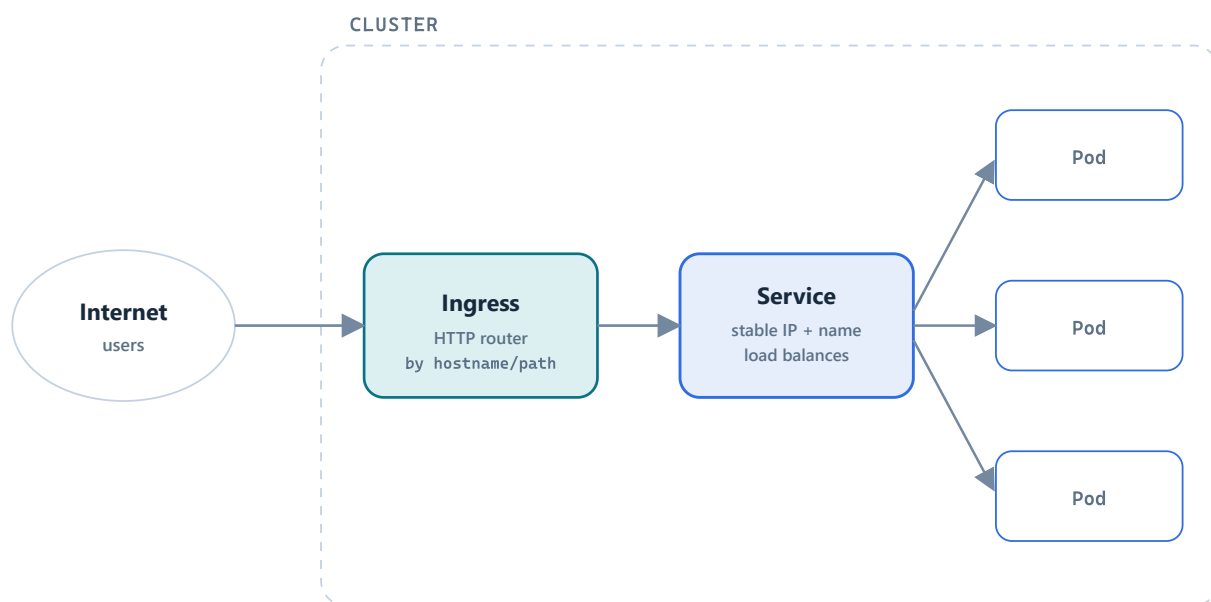


Figure 4 – The path of a request. A user's request hits the **Ingress** (an HTTP router that maps hostnames/paths to Services), which forwards to a **Service**, which load-balances across the live **Pods**. OpenShift's **Route** plays the Ingress role with a simpler, built-in experience.

ClusterIP default

Reachable only *inside* the cluster. The default.
Perfect for a backend that only other Pods call.

NodePort service

Opens the same port on every node's IP, so outside traffic can reach the Service. Simple but low-level.

LoadBalancer service

Asks the cloud provider for a real external load balancer with a public IP. The usual way to expose a Service in the cloud.

Ingress router

Not a Service type – a smart HTTP(S) router in front of Services. Handles hostnames, paths, and TLS for many apps through one entry point.

i **Also good to know: the flat network model.** In Kubernetes, every Pod gets its own IP and every Pod can reach every other Pod directly, across nodes, with no address translation. This is enforced by a *CNI plugin* (Calico, Cilium, and others). **NetworkPolicies** then let you restrict who may talk to whom.

PART 06 Configuration & storage

Good containers are generic and reusable. You don't bake settings or passwords into the image — you inject them at run time. And because containers are disposable, any data that must survive needs somewhere permanent to live.

ConfigMaps & Secrets — settings without rebuilds

ConfigMap

Holds **non-sensitive** configuration — feature flags, URLs, log levels — as key/value pairs. Injected into Pods as environment variables or files. Change the config without touching the image.

Secret

Same idea, but for **sensitive** values — passwords, API keys, TLS certs. Stored separately and handled more carefully (base64-encoded, access-controlled, and can be encrypted at rest).



Secrets are not encrypted by default — base64 is *encoding*, not encryption. On a real cluster, turn on encryption at rest and restrict access with RBAC (role-based access control). OpenShift tightens several of these defaults for you.

Volumes, PV & PVC — data that outlives the Pod

When a container restarts, everything it wrote to its own filesystem is gone. For data that must persist — a database's files, uploaded documents — Kubernetes uses **persistent storage**, described in three parts:

- **PersistentVolume (PV)** — a piece of real storage in the cluster (a cloud disk, an NFS share). Think of it as the *actual* hard drive.
- **PersistentVolumeClaim (PVC)** — a Pod's *request* for storage: "I need 10 GB, read-write." Kubernetes binds the claim to a matching PV.
- **StorageClass** — a template for creating PVs on demand (*dynamic provisioning*), so you don't hand-create disks. "Give me fast SSD storage" and one appears.

THE MENTAL MODEL

A **PVC is a claim ticket at a coat check**. You (the Pod) hand over a ticket asking for storage; the system finds an actual coat rack slot (the PV) and binds them. Your Pod can be replaced, but as long as the claim exists, it gets the same data back.

PART 07 Talking to the cluster with `kubectl`

`kubectl` ("cube control") is the command-line tool for Kubernetes. It sends your requests to the API Server. There are two styles of using it, and the difference matters.

Declarative (recommended)

You write YAML files describing the desired state and apply them. Repeatable, reviewable, version-controlled. This is how real teams work â€” it's the foundation of *GitOps*.

● DECLARATIVE

```
kubectl apply -f app.yaml
```

Imperative (quick)

You issue direct commands that change things immediately. Great for learning and experiments, poor for anything you need to reproduce later.

● IMPERATIVE

```
kubectl create deployment web \
  --image=my-web-app:2.0
```

● THE COMMANDS YOU'LL TYPE CONSTANTLY

```
# See what's running
kubectl get pods           # list Pods in the current namespace
kubectl get pods -A        # ...across all namespaces
kubectl get deployments,svc # list several kinds at once

# Investigate one thing in detail
kubectl describe pod web-xyz # events, status, why it won't start
kubectl logs web-xyz         # the container's logs
kubectl logs -f web-xyz      # ...follow live (tail)

# Act on the cluster
kubectl apply -f app.yaml   # create/update from a file
kubectl scale deploy/web --replicas=5
kubectl exec -it web-xyz -- sh # open a shell inside a container
kubectl delete -f app.yaml   # remove what the file defined
```

i The debugging trio. When something's wrong, 90% of the time you reach for `get` (what exists?), `describe` (what's its status and recent events?), and `logs` (what did the app itself say?). Learn these three cold.

PART 08 Enter OpenShift

Everything so far is pure Kubernetes â€” powerful, but deliberately unopinionated. It's a kit of parts, not a finished product. You still have to choose and assemble networking, security, registries, CI/CD, monitoring, and more. **OpenShift** is Red Hat's answer to "just give me the whole thing, secured and supported."

OpenShift is Kubernetes at its core â€” a certified Kubernetes distribution. It doesn't replace Kubernetes; it **packages** it with a curated, integrated set of enterprise tools, sensible secure defaults, and commercial support. If Kubernetes is an engine, OpenShift is the whole car: engine included, plus the chassis, dashboard, safety systems, and a warranty.

WHAT DEVELOPERS & ADMINS TOUCH

Web Console

rich graphical UI

oc CLI & developer tools

kubectl + extra commands

OpenShift platform services

Routes Â· integrated registry Â· S2I builds Â· Operators
stricter security (SCC) Â· monitoring Â· logging Â· CI/CD

Kubernetes

the orchestration core â€” unchanged

Red Hat Enterprise Linux / RHCOS

Figure 5 â€” The OpenShift stack. Kubernetes sits unchanged at the core, on Red Hat's OS. OpenShift adds an integrated platform layer (in **red**) and polished tools on top. Anything you can do in plain Kubernetes still works â€” OpenShift only adds.

â–†

The flavors of OpenShift. *OKD* is the free, community upstream. *OpenShift Container Platform (OCP)* is the self-managed commercial product. *ROSA / ARO / OpenShift Dedicated* are fully-managed cloud services (on AWS, Azure, etc.). Same platform, different amounts of "who runs it for you."

PART 09 What OpenShift adds on top

These are the concepts you'll meet in OpenShift that don't exist (or aren't built-in) in vanilla Kubernetes. Each one maps to a Kubernetes idea you already know.

Projects “ namespaces with guardrails

A **Project** is an OpenShift *Namespace* plus additional annotations, default access controls, and self-service rules. In practice, developers are given permission to create their own Projects, each an isolated workspace with quotas and role bindings applied automatically. It's the multi-tenancy story made turnkey.

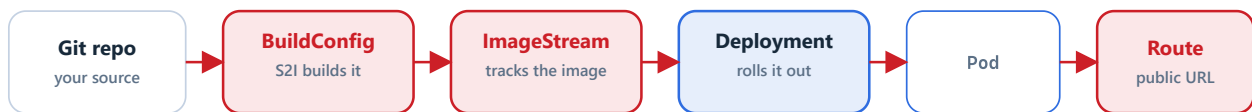
Routes “ exposing apps the easy way

Where Kubernetes gives you Ingress (which you must install and configure a controller for), OpenShift ships a built-in router and a simpler object called a **Route**. Create a Route and OpenShift immediately gives your Service a public hostname with TLS handling built in “ often a single click or command. Same job as Ingress, far less assembly.

Source-to-Image (S2I) & BuildConfigs “ code straight to container

This is one of OpenShift's signature features. With plain Kubernetes, *building* a container image is your problem “ you set up Docker, write a Dockerfile, run a CI pipeline, push to a registry. OpenShift can do it inside the cluster: point a **BuildConfig** at your Git repository and OpenShift's **Source-to-Image** process compiles your source, produces a runnable image, and deploys it “ no Dockerfile required.

PUSH CODE → RUNNING, PUBLIC APP ← ALL INSIDE THE CLUSTER



A push to Git can even re-trigger the whole chain automatically.

Figure 6 – Source-to-Image. OpenShift can take you from a Git repository to a live, publicly-reachable app entirely inside the cluster – building the image, tracking it, deploying it, and exposing it – with no external CI system or Dockerfile required.

ImageStreams – a smart pointer to images

An **ImageStream** is an OpenShift abstraction over container images. Instead of hard-coding `registry.example.com/app:2.0` everywhere, your Deployments reference the ImageStream. When a new image is built or pushed, the ImageStream updates and can automatically trigger a redeploy. It decouples "what image is current" from "who's using it."

Operators – packaged operational knowledge

An **Operator** extends Kubernetes with custom logic to manage a complex application (a database, a message broker) the way a human operator would – handling install, upgrades, backups, and failure recovery automatically. Operators are a Kubernetes concept, but OpenShift makes them central: it ships with an *OperatorHub*, a built-in catalog where you install these capabilities with a click. Much of OpenShift itself is built from Operators.

Security defaults – safe out of the box

Vanilla Kubernetes lets containers run as the *root* user by default, which is risky. OpenShift is **secure by default**: through **Security Context Constraints (SCC)**, it refuses to run containers as root unless you explicitly allow it, and assigns each Project a random user ID. This trips up beginners porting images from Docker – but it's the platform protecting you. It also integrates authentication, RBAC, and an internal image registry from day one.

The `oc` CLI & Web Console

OpenShift's command-line tool is `oc`. It's a superset of `kubectl` â€” every `kubectl` command works as `oc`, plus extras like `oc new-app` (build and deploy in one step), `oc login`, and `oc new-project`. Alongside it, OpenShift ships a genuinely polished **web console** with separate Developer and Administrator perspectives â€” something plain Kubernetes leaves to third-party dashboards.

PART 10 Kubernetes vs. OpenShift, side by side

They are not competitors – OpenShift *contains* Kubernetes. The real question is "raw platform you assemble yourself" versus "integrated product with support." This table maps the concepts you now know.

CONCEPT	KUBERNETES (VANILLA)	OPENSIFT
What it is	Open-source orchestration core; a kit of parts	A certified Kubernetes <i>distribution</i> – the kit, pre-assembled
Isolation unit	Namespace	Project (Namespace + policy & self-service)
External access	Ingress (install a controller yourself)	Route (built-in router, ready to use)
Building images	Your responsibility (external CI, Dockerfile)	S2I / BuildConfig builds inside the cluster
Image tracking	Reference registry paths directly	ImageStreams abstract & trigger redeploys
CLI	kubectl	oc (superset of kubectl)
Web UI	Basic dashboard, optional add-on	Rich console, Dev + Admin perspectives, built-in
Security default	Containers may run as root	Non-root enforced via SCC; secure by default
Registry	Bring your own	Integrated internal registry included
Support	Community (or your cloud provider's)	Commercial Red Hat support & lifecycle
Cost	Free (you pay for infra & effort)	Subscription (OKD is the free upstream)



The one-line summary. Learn *Kubernetes* concepts first – they are universal and everything transfers. OpenShift is Kubernetes with the tedious assembly, the security hardening, and the support contract already done for you. Skills move freely in both directions.

PART 11 **Command cheat sheet**

Since `oc` is a superset of `kubectl`, nearly everything on the left also works on the right. The right column shows OpenShift's *extra* conveniences.

kubectl • Kubernetes

● EVERYDAY KUBECTL

```
kubectl get pods
kubectl get all
kubectl describe pod NAME
kubectl logs -f NAME
kubectl apply -f file.yaml
kubectl scale deploy/web --replicas=3
kubectl exec -it NAME -- sh
kubectl create namespace dev
kubectl config use-context CTX
kubectl delete -f file.yaml
```

oc • OpenShift

● OC ADDS THESE ON TOP

```
oc login https://api.cluster:6443
oc new-project dev
oc project dev
oc new-app python:3~https://git/repo
oc expose svc/web           # make a
Route
oc get routes
oc start-build web
oc status
oc whoami
oc get pods                 # kubectl
still works
```

PART 12 Your learning path from here

Reading gets you the map. You learn this by doing. Here's a concrete, low-cost order that builds real intuition.

1 Get comfortable with containers first.

Install Docker or Podman. Build an image from a `Dockerfile`, run it, push it to a registry. Kubernetes makes no sense until containers are second nature.

2 Run a real cluster on your laptop.

Use [minikube](#), [kind](#), or k3s. Deploy the Deployment → Service → Ingress chain from this guide by hand.

3 Write YAML from scratch.

Create a Deployment, a Service, a ConfigMap, and a PersistentVolumeClaim yourself. Break them on purpose and use `describe` / `logs` to diagnose.

4 Master the debugging trio.

`get`, `describe`, `logs`. Being able to answer "why won't this Pod start?" is the single most valuable operational skill.

5 Try OpenShift free.

Spin up [OpenShift Local \(CRC\)](#) or use the time-limited *Developer Sandbox*. Do an `oc new-app` from Git and watch S2I build and deploy it.

6 Then go deeper.

Helm charts (packaging), Operators (automation), RBAC (security), monitoring with Prometheus/Grafana, and GitOps with Argo CD. Each is a natural next chapter.



Certifications worth knowing. The CKA (Certified Kubernetes Administrator) and CKAD (Application Developer) are the standard Kubernetes credentials. Red Hat's EX280 covers OpenShift administration. All three are hands-on, terminal-based exams — which is exactly why the practice above matters.

Cluster

A set of machines (nodes) that Kubernetes manages as one system.

Node

A single machine (physical or virtual) in the cluster. Control-plane nodes decide; worker nodes run your containers.

Pod

The smallest deployable unit – one or more tightly-coupled containers sharing network and storage.

Deployment

Manages a replicated, self-healing set of identical Pods, with rolling updates and rollbacks.

ReplicaSet

Ensures a specified number of Pod copies are always running. Usually managed by a Deployment.

Service

A stable network address and load balancer in front of a changing set of Pods.

Ingress

An HTTP(S) router mapping external hostnames/paths to Services. OpenShift's equivalent is a Route.

Namespace

A virtual partition of a cluster for isolation and access control. Called a Project in OpenShift.

ConfigMap / Secret

Inject non-sensitive / sensitive configuration into Pods without rebuilding images.

PV / PVC

PersistentVolume is real storage; PersistentVolumeClaim is a Pod's request to use some, enabling data to outlive Pods.

etcd

The key-value database holding the cluster's entire state – its single source of truth.

kubelet

The agent on every node that runs and monitors the Pods assigned to it.

Operator

Custom automation that manages a complex app (install, upgrade, recover) the way a human operator would.

Route (OpenShift)

OpenShift's built-in way to expose a Service at a public hostname with TLS – like Ingress, but ready out of the box.

S2I / BuildConfig (OpenShift)

Source-to-Image: build a container image from source code inside the cluster, no Dockerfile needed.

ImageStream (OpenShift)

A managed pointer to container images that can trigger redeploys when the image changes.

SCC (OpenShift)

Security Context Constraints – policies that enforce secure defaults, e.g. no running as root.

kubectl / oc

The command-line tools for Kubernetes and OpenShift respectively; `oc` is a superset of `kubectl`.

This guide covers the conceptual foundation of Kubernetes and OpenShift for beginners. Everything here is stable, transferable knowledge – the specifics of any one cluster will vary, but these ideas are the ground you'll always stand on. Now go build something and break it. [Fair winds.](#) 🍃